BPM
CSE345
PETRI NETS

Dr Islam El-Maddah

Petri Nets

- Petri nets have proven to be a particularly effective mechanism for modeling the **dynamic aspects of processes**.

- Petri Nets they have three specific advantages:
  - Formal semantics despite the graphical nature.
  - State-based instead of event-based.
  - Abundance of analysis techniques.

Petri Nets

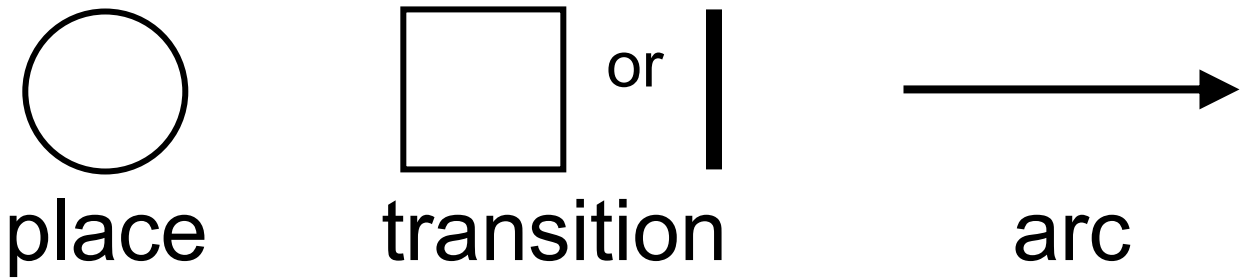
- Originate from C.A. Petri's PhD thesis (1962).
- They were originally conceived as a technique for the description and *analysis of concurrent behaviour in distributed systems*.
- Based on a few simple concepts, yet expressive.
- They have a **simple graphical format** and a **precise operational semantics** that makes them an attractive option for modeling the static and dynamic aspects of processes.
- Many analysis techniques exist.
- Many extensions and variants have been defined over the years.

Applications

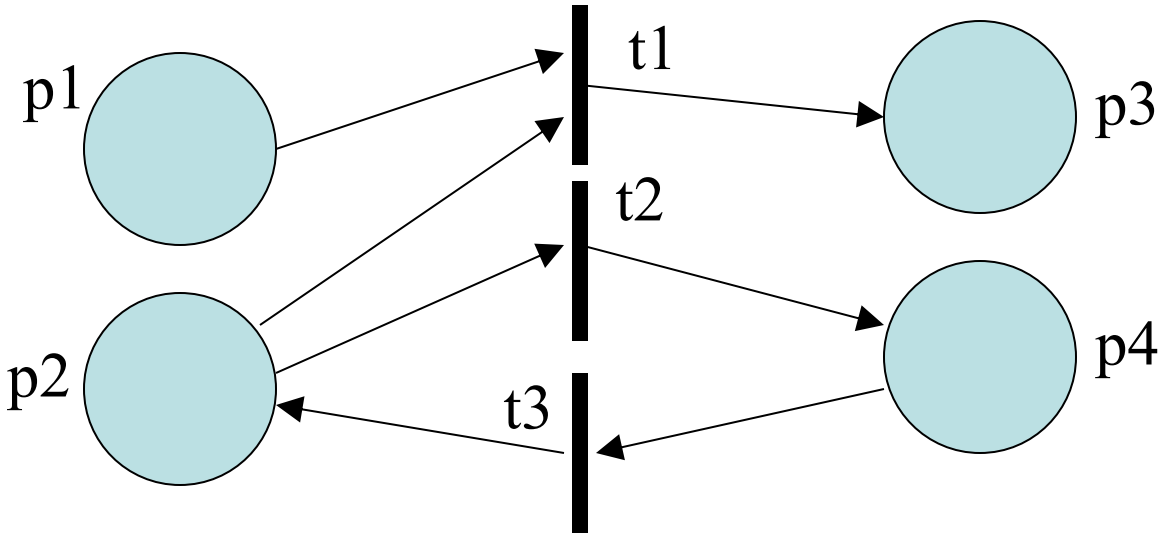
- Applications in many different areas, such as databases, software engineering, formal semantics, etc.
- There are two main uses of Petri nets for workflows:
  - Specifications of workflows.
  - Formal foundation for workflows (semantics, analysis of properties).

Petri Nets: Basics

- A Petri Net takes the form of a **directed bipartite graph** where the nodes are either *places* or *transitions.*
- **Places** represent **intermediate states** that may exist during the operation of a process. Places are represented by circles.
- Places can be input/output of **transitions**. Transitions correspond to the **activities** or **events** of which the process is made up. Transitions are represented by rectangles or thick bars.
- **Arcs** connect places and transitions in a way that places can only be connected to transitions and vice-versa.



Petri Net: Example



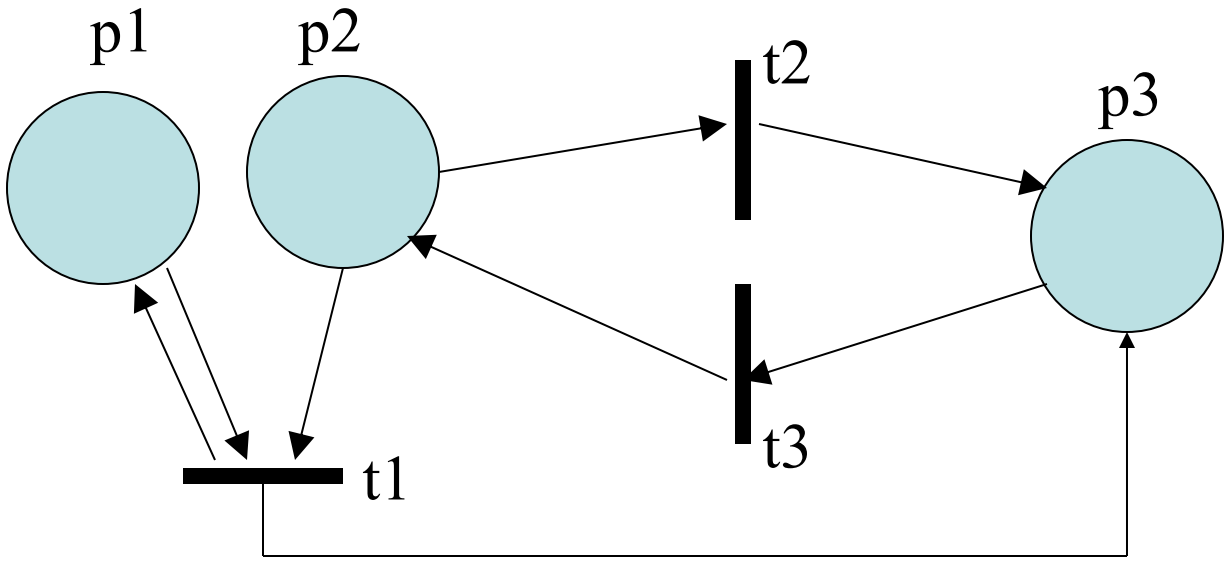
$P = \{p_1, p_2, p_3, p_4\}$

$T = \{t_1, t_2, t_3\}$

$F = \{(p_1, t_1), (p_2, t_1), (t_1, p_3), (p_2, t_2), (t_2, p_4), (p_4, t_3), (t_3, p_2)\}$

$t_1 \bullet = \{p_3\}$; $\bullet t_1 = \{p_1, p_2\}$; $\bullet p_2 = \{t_3\}$; $\bullet p_1 = \varnothing$; $p_2 \bullet = \{t_1, t_2\}$

Petri Nets: Example



P = ...

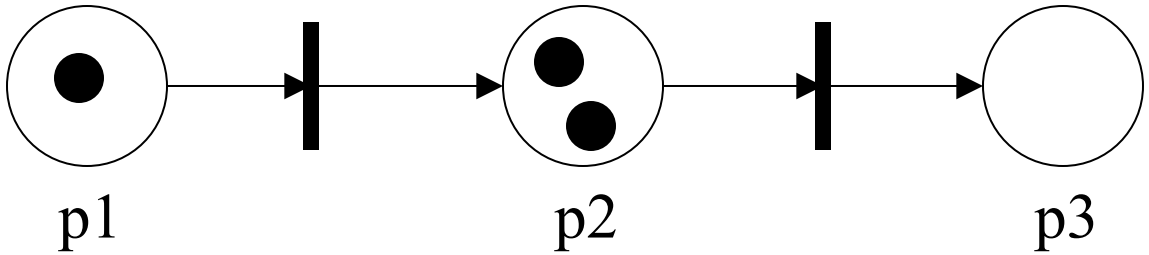
T = …

F = ...

$t_1 \bullet$ = …… ; $\bullet t_1$ = …… ; $\bullet p_2$ = …… ; $p_2 \bullet$ = ……

Markings

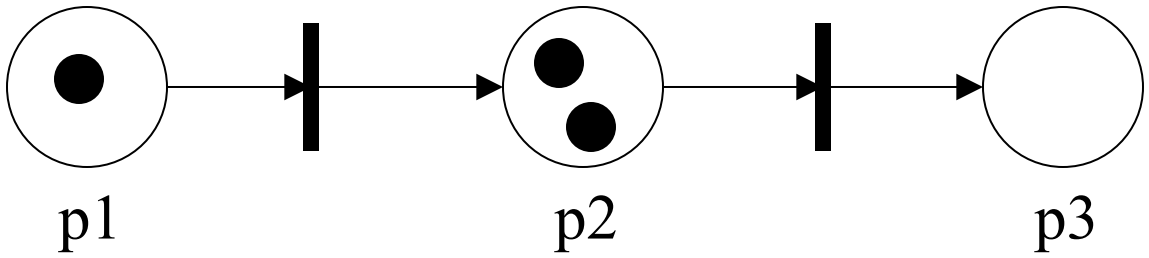
- The operational semantics of a Petri Net is described in terms of particular marks called ***tokens*** (graphically represented as black dots ●).
- Places in Petri Nets can contain any number of tokens. The distribution of tokens across all of the places in a net is called a **marking**. For a Petri net an *initial marking* M_0 needs to be specified.
- Marking assigns tokens to places; formally, a marking M of a Petri net N = (P,T,F) is a function **M: P -> NAT**.



- The marking below is formally captured by the following marking M = {(p_1,1),(p_2,2),(p_3,0)}.

State of a Petri Net

- A state can be compactly described as shown in the following example:
  - **$1p_1+2p_2+0p_3$** is the state with one token in place p1, two tokens in p2 and no tokens in p3.
  - We can also represent this state in the following (equivalent) way: **p_1+2p_2**.

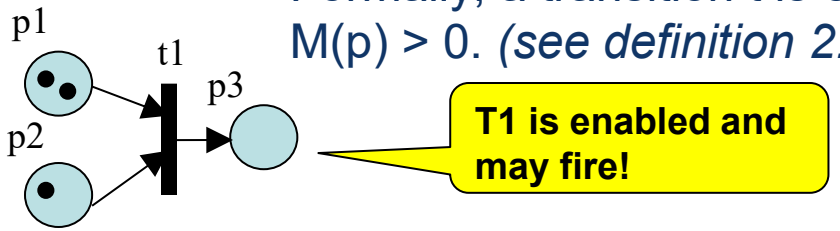


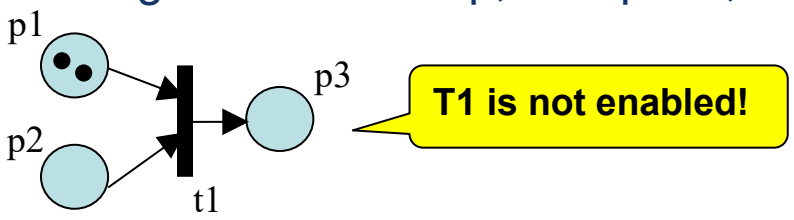
- We can also describe an ordering function **≥** over the set of possible states such that, given a Petri net N = (P,T,F) and markings M and M′, **M ≥ M′** iff for all p in P: **M(p) ≥ M′(p).** M > M′ iff M ≥ M′ and M ≠ M′.

Enabled Transitions

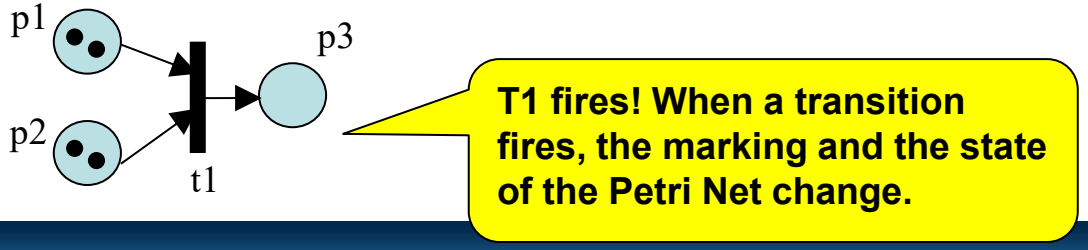
- The operational semantics of Petri nets are characterized by the notion of a transition executing or "**firing**". A transition in a Petri net can "fire" whenever there are one or more tokens in each of its input places.
- The execution of a transition occurs in accordance with the following firing rules:

1. A transition t is said to be **enabled** if and only if each input place p of t contains at least one token. Only enabled transitions may fire.
   - Formally, a transition t is enabled in a marking M iff for each p, with p $\in$•t, M(p) > 0. *(see definition 2.7 of [DE95])*

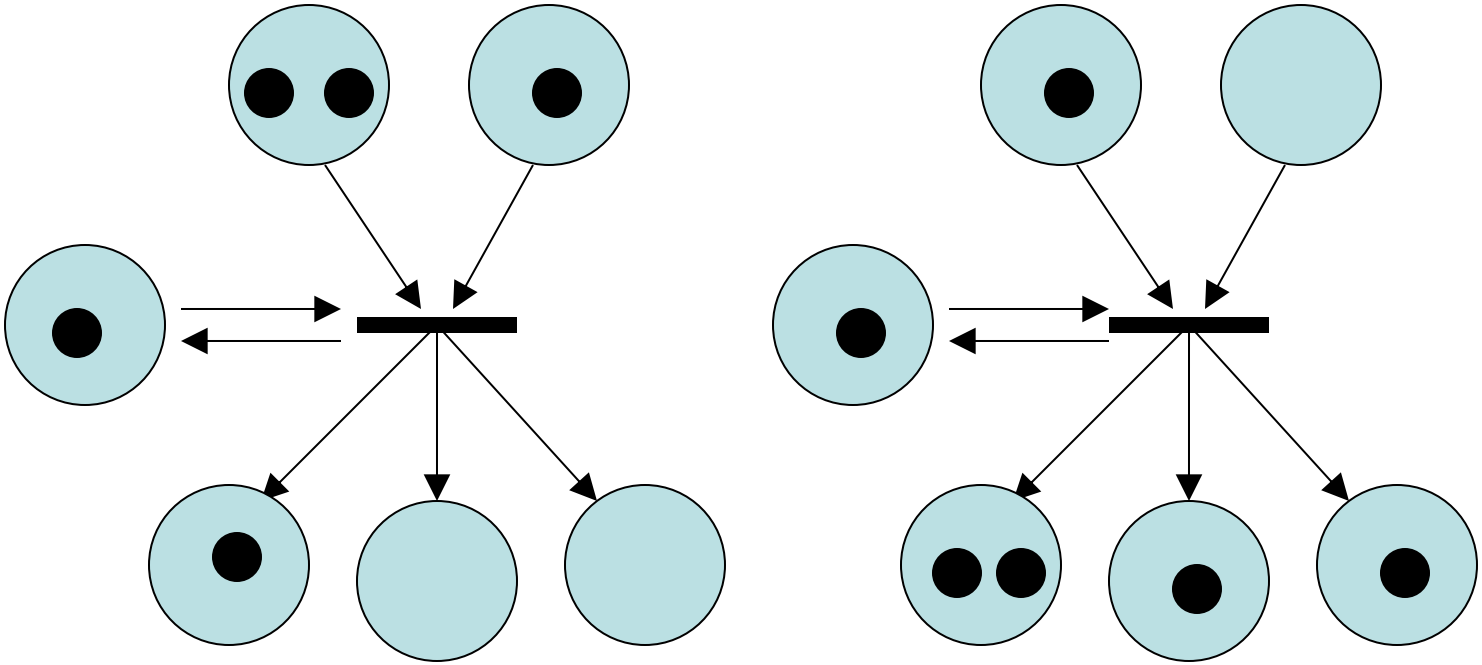




2. If transition t fires, then t **consumes one token** from each input place p of t and **produces one token** for each output place p of t .



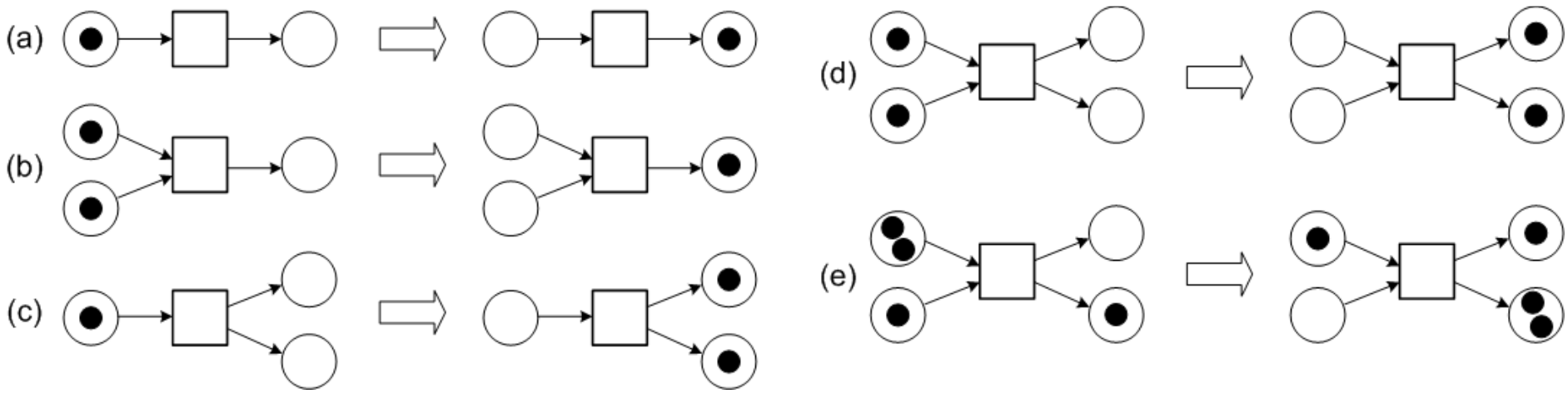
Firing a Transition: Example



BEFORE

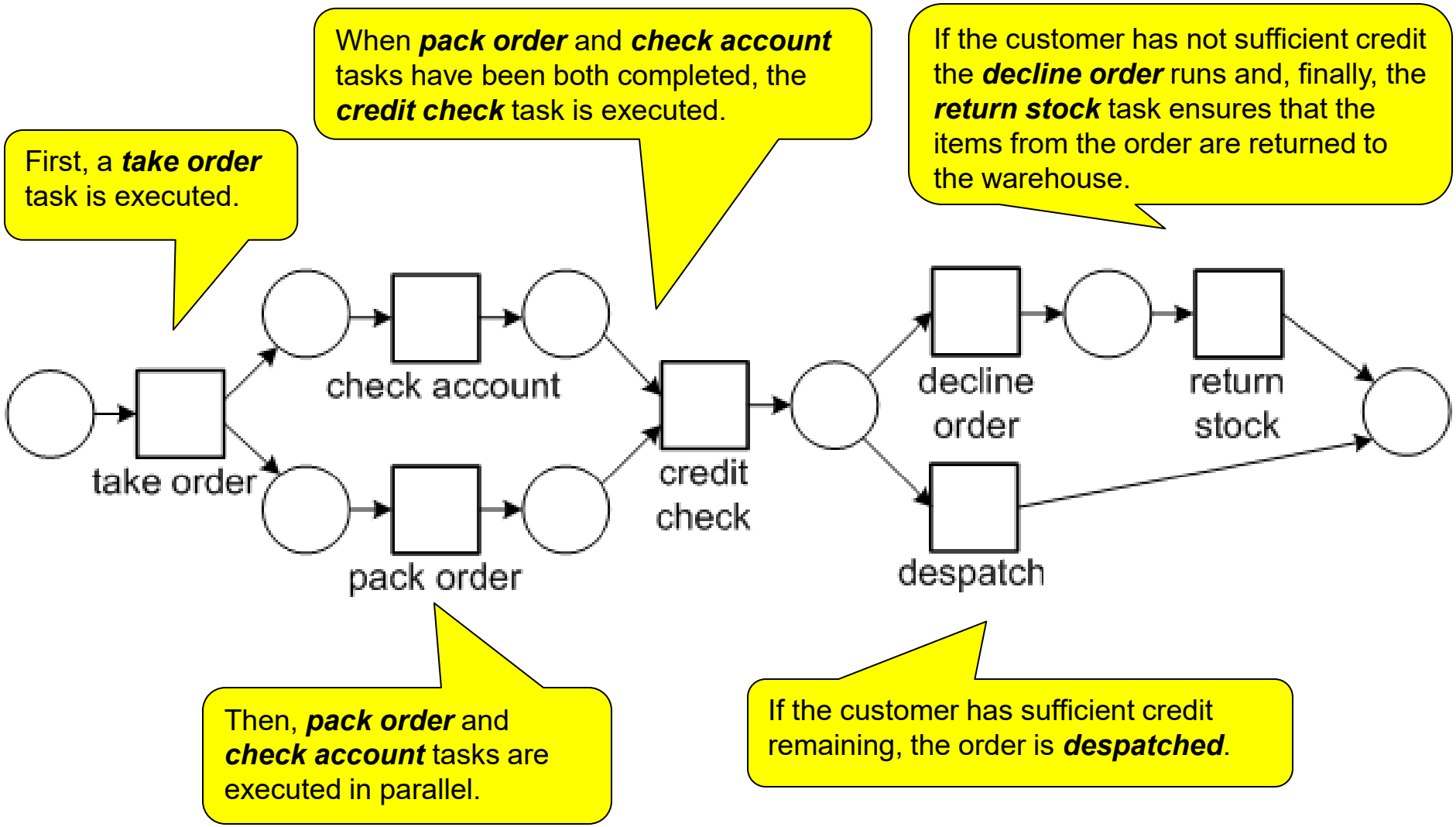
AFTER

Firing Transitions: Further Examples



- It is assumed that the firing of a transition is an atomic action that occurs instantaneously and cannot be interrupted.
- If there are multiple enabled transitions, any one of them may fire; however, for execution purposes, it is assumed that **they cannot fire simultaneously.**
- An enabled transition is not forced to fire immediately but can do so at a time of its choosing.
- These features make Petri nets particularly suitable for modeling concurrent process executions.

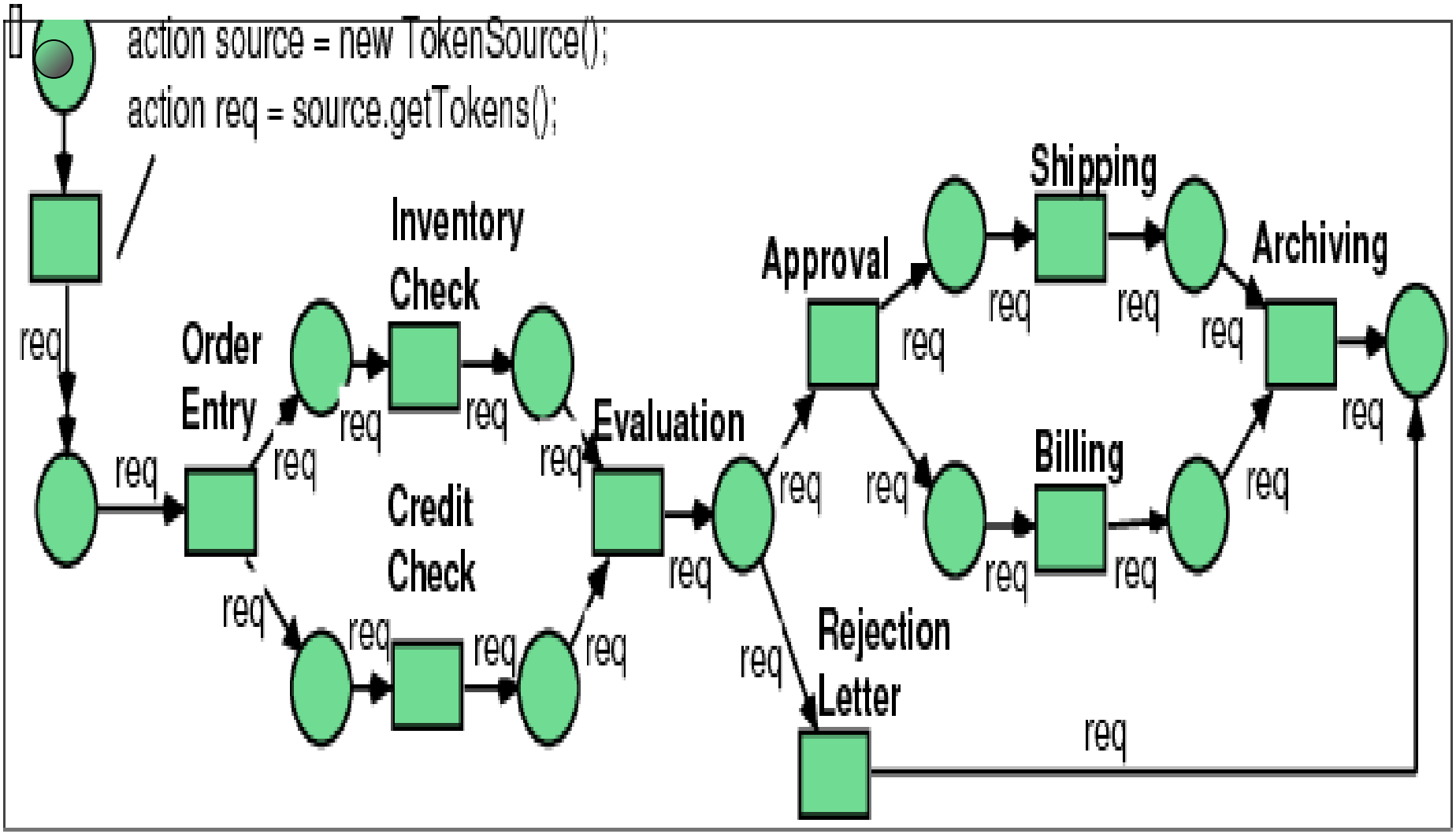
Petri nets: Order Fulfillment Example



Why petri net execution is useful

#	Reason	Why It Is Important
1	Verify Process Correctness	Ensures the process can complete correctly and reach the final state.
2	Detect Modeling Errors	Helps find problems like deadlocks, livelocks, or unreachable tasks.
3	Simulate Process Behavior	Shows how tokens (cases) move through the process like real executions.
4	Performance Analysis	Enables measurement of cycle time, throughput, WIP, and bottlenecks.
5	Process Automation	Allows the model to run directly in workflow or business process systems.

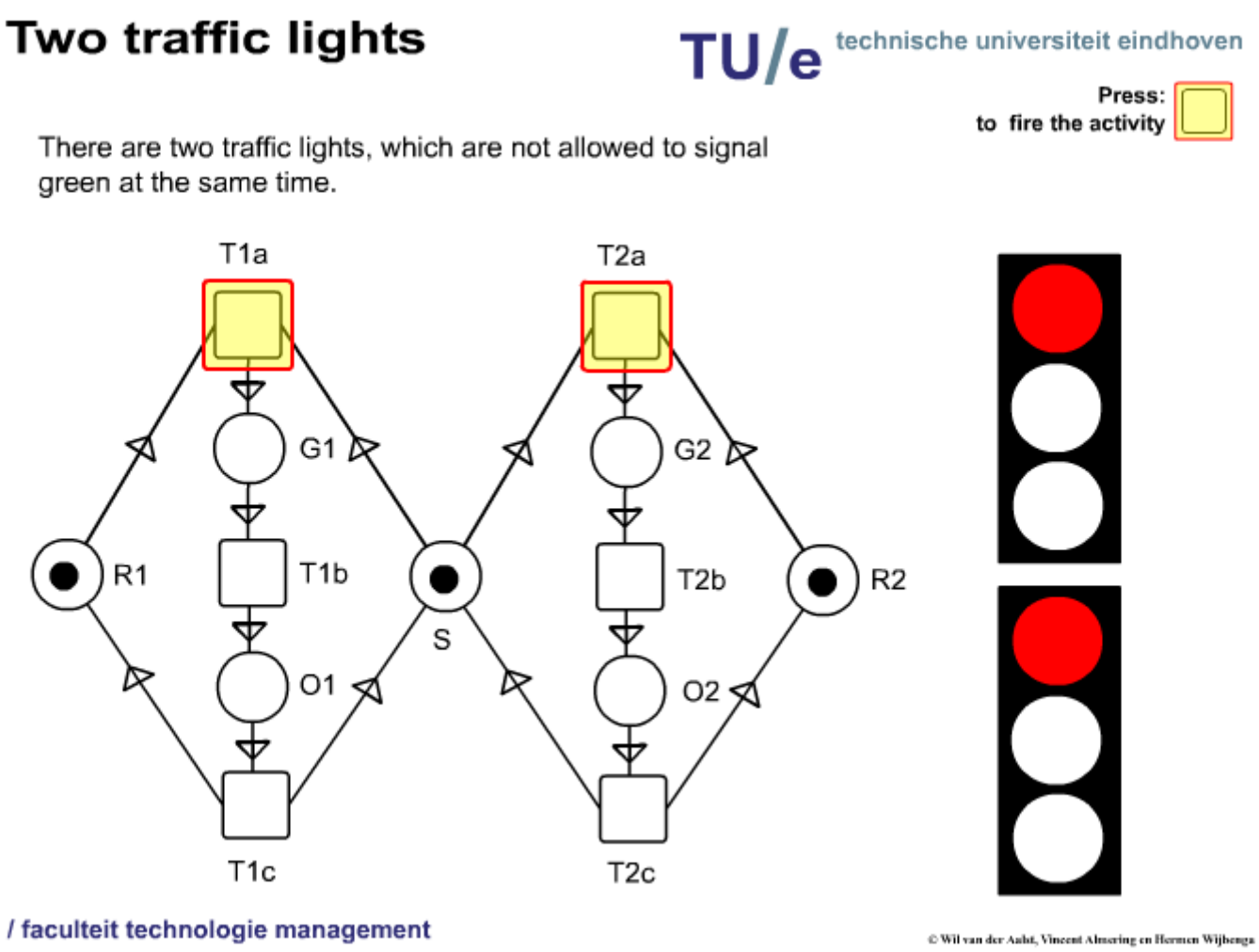
Detailed Petrinet



Modelling Exercise

- We want to model with a Petri Net the behaviour of two traffic lights at an intersection, in a way that they cannot be green or yellow at the same time.
- Conversely, they are allowed to signal red at the same time.

Solution Traffic Lights

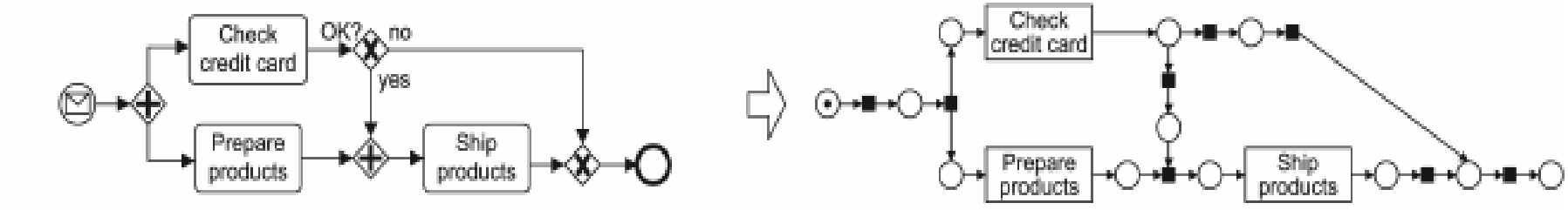


Animation by Wil van der Aalst, Vincent Almering and Herman Wijbenga

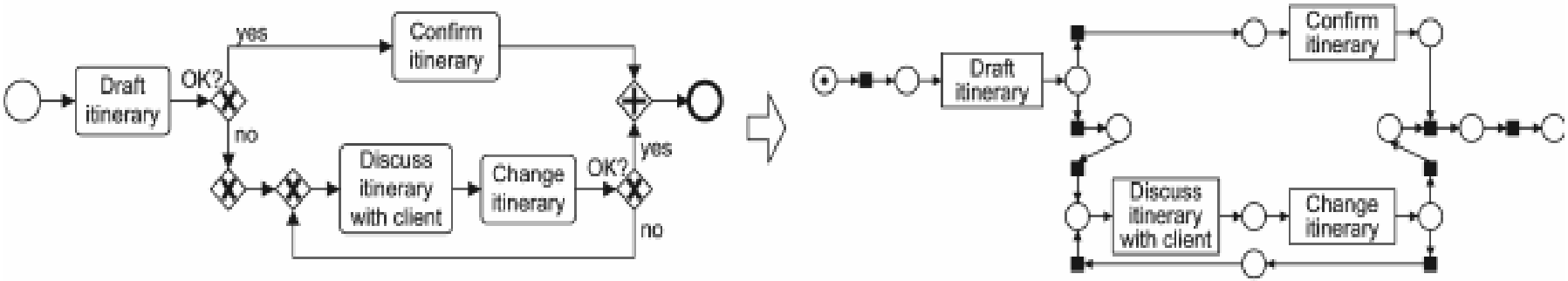
Homeworks

- Two traffic lights at an intersection. If one is red, the other should be green etc. *(many discussions on modelling traffic lights through Petri nets can be found on the internet).*
- A producer and a consumer producing and consuming (resp.) indefinitely. The consumer cannot consume more than the producer has produced thus far. How does your model change if the buffer between them is of limited size? *(this is a well-known concurrency problem)*
- Two parallel processes with two critical sections. If one of the two processes is in its critical section, the other process should not be able to enter its critical section and vice versa. *(this is also a well-known concurrency problem)*

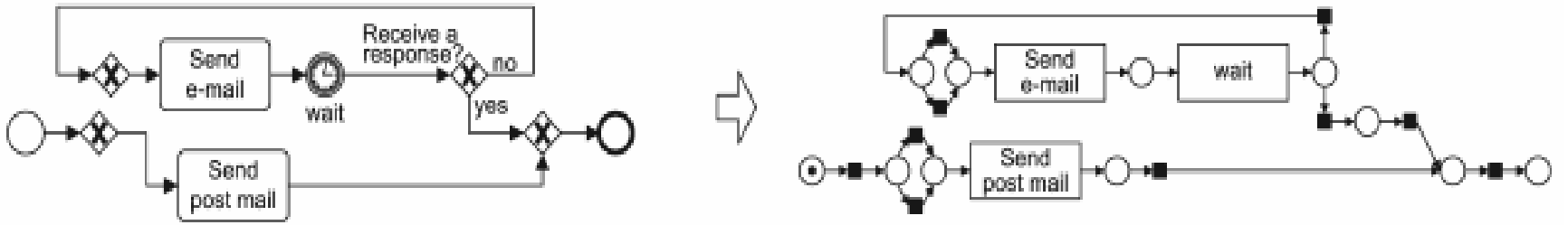
BPMN to Petrinet



(a) Order process

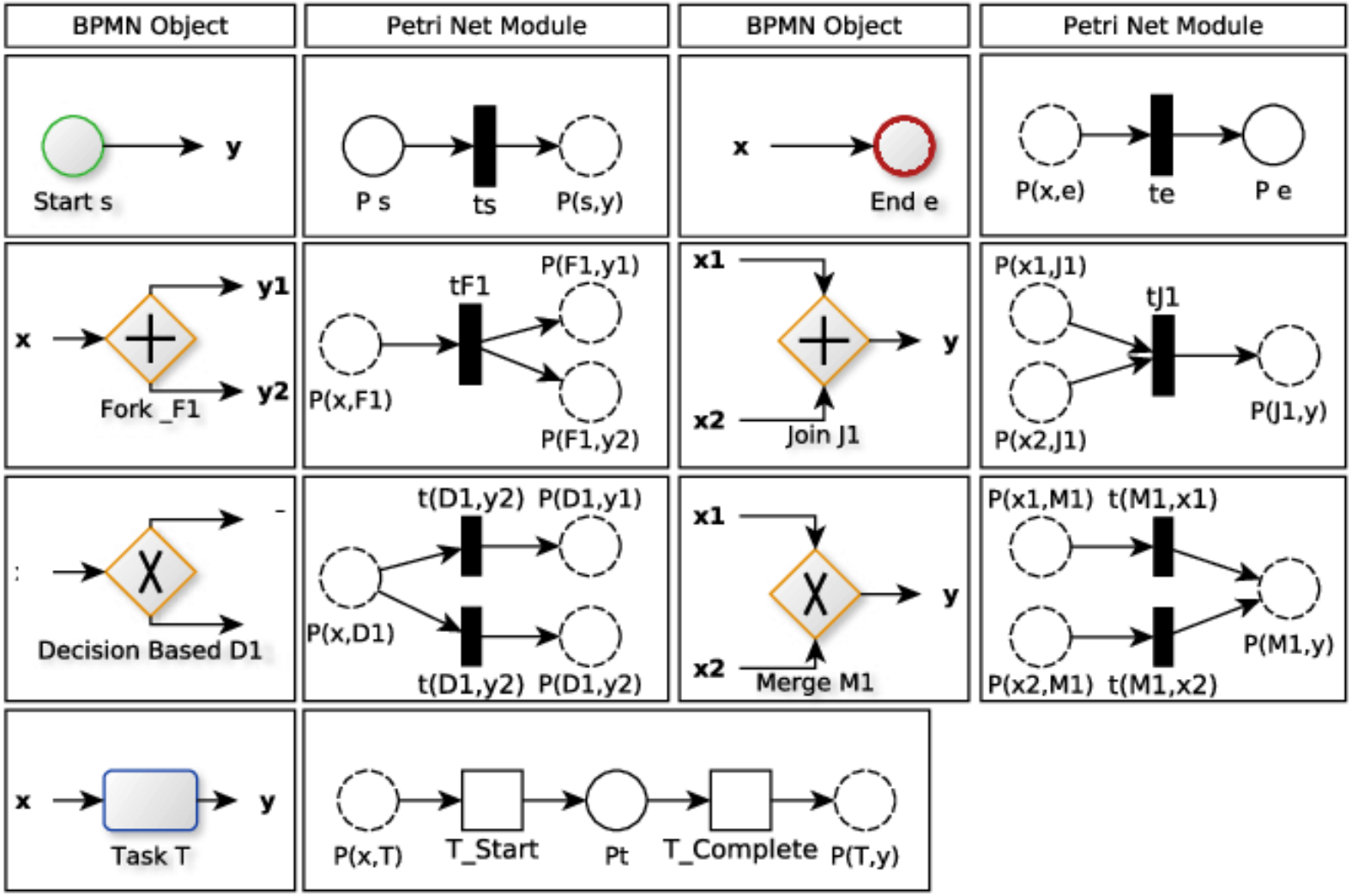


(b) Travel itinerary process



(c) Answer process

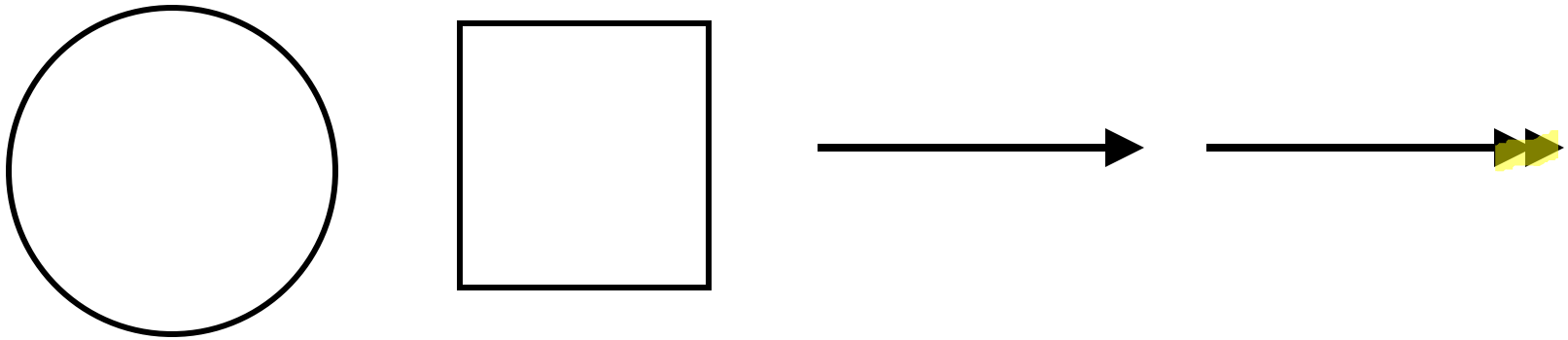
Note: ■ silent transition


BPMN Object
Petri Net Module
BPMN Object
Petri Net Module
Start s
y
P s
ts
P(s,y)
x
End e
P(x,e)
te
P e
x
y1
y2
Fork _F1
tF1
P(F1,y1)
P(x,F1)
P(F1,y2)
x1
x2
y
Join J1
P(x1,J1)
tJ1
P(x2,J1)
P(J1,y)
t(D1,y2)
P(D1,y1)
P(x,D1)
t(D1,y2)
P(D1,y2)
Decision Based D1
x1
x2
y
Merge M1
P(x1,M1)
t(M1,x1)
P(x2,M1)
t(M1,x2)
P(M1,y)
x
y
Task T
P(x,T)
T_Start
Pt
T_Complete
P(T,y)

Reset nets

- While Petri nets and Workflow nets provide an effective means of modeling a business process, **they are not able to capture the notion of cancelation**.
  - This is a significant omission, given the relative frequency of cancelation in real-life processes.
- **Reset nets** extend Petri nets with a special type of arc, the ***reset arc***, to capture the notion of cancelation.
- Like normal input arcs, reset arcs **connect places to transitions**.
- However, they operate in a different way: when a transition to which a reset arc is connected fires, **any place connected to the transition by reset arcs is emptied of any tokens that it may contain**.
- Note that the tokens in places attached to a transition by reset arcs **play no part in the enablement of the transition**.
  - This makes reachability **undecidable**.

Reset nets

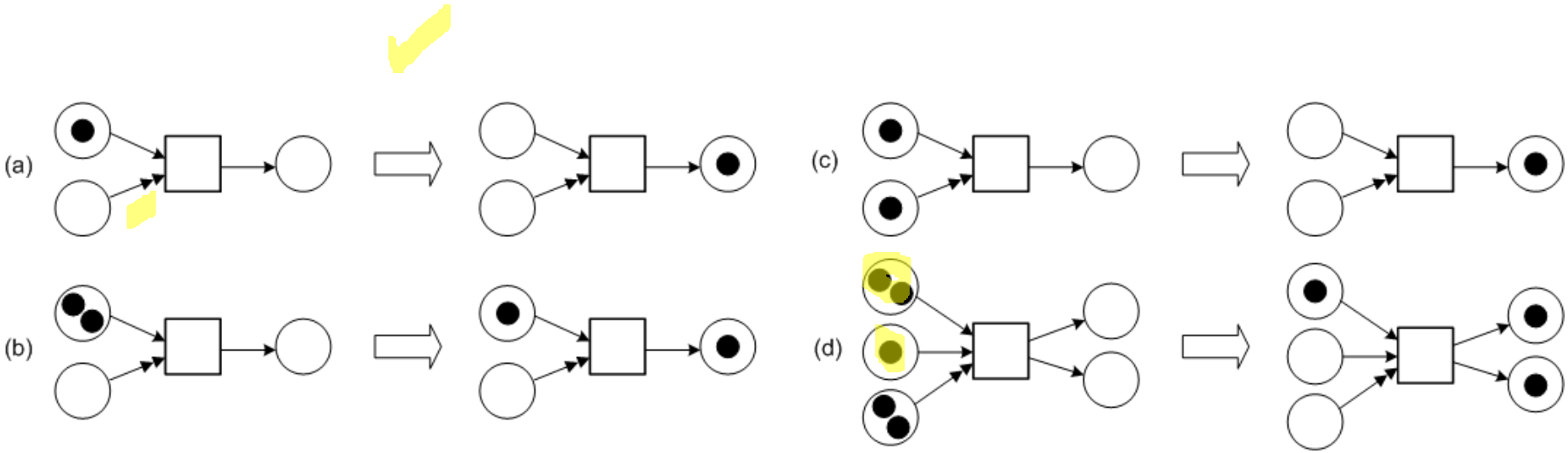


place transition arc reset arc

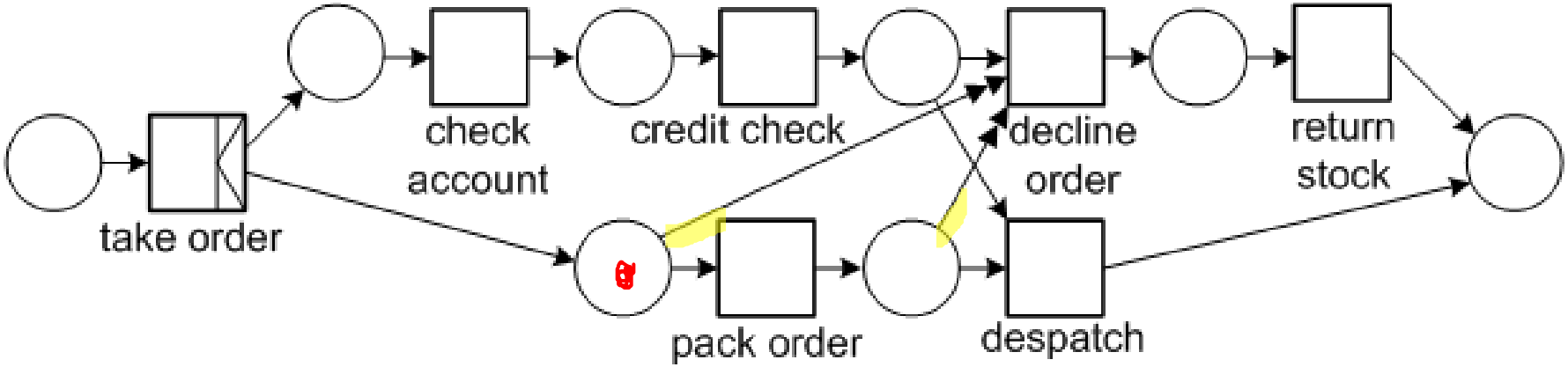
Reset nets: formal definition

- Formal definitions are based on [DFS98, FRSB02, FS01].
- Syntactically a **Reset net** is a tuple (P, T, F, R) where
  - (P, T, F) is a Petri net with a finite set of places P, a finite set of transitions T , and a flow relation $F \subseteq (P \times T \cup T \times P)$;
  - $R: T \rightarrow 2^P$ is a function associating reset places with transitions.
- A reachable marking M' is defined by:
  - first removing the tokens needed to enable t from its input places $\bullet t$;
  - then removing all tokens from reset places associated with t (i.e., R(t));
  - and finally by adding tokens to its output places $t\bullet$.
- Let N = (P, T, F, R) be a Reset net and M a marking.
  - A transition $t \in T$ is enabled iff $\bullet t \leq M$.
  - An enabled transition t can fire thus changing the state to M', denoted $M \rightarrow^t M'$, with $M' = (M - \bullet t)[P \backslash R(t)] + t\bullet$.
- The definition of occurrence sequence extends naturally from Petri nets.

Reset nets



Reset nets: the order fulfillment process



Sources and References

- [Aalst96] Wil M. P. van der Aalst. Three Good Reasons for Using a Petri-net-based Workflow Management System. In S. Navathe and T. Wakayama, editors, Proceedings of the International Working Conference on Information and Process Integration in Enterprises (IPIC'96), pages 179-201, Cambridge, Massachusetts, November 1996.
- [Aalst97] Wil M.P. van der Aalst. Verification of Workflow Nets. In P. Azéma and G. Balbo, editors, Applications and Theory of Petri Nets 1997, volume 1248 of Lecture Notes in Computer Science, pp 407-426, Springer Verlag, 1997.
- [AH02] Wil M.P. van der Aalst and Kees M. van Hee. Workflow Management: Models, Methods, and Systems. The MIT Press, 2002.
- [JB96] S. Jablonski and C. Bussler. Workflow Management: Modeling Concepts, Architecture and Implementation. International Thomson Computer Press, 1996.
- [BW90] J. Baeten and W.P. Weijland. Process Algebra. Cambridge Tracts in Theoretical Computer Science 18, Cambridge University Press, 1990.
- [DE95] J. Desel and J. Esparza. Free Choice Petri Nets. Cambridge Tracts in Theoretical Computer Science 40, Cambridge University Press, 1995.
- [DFS98] C. Dufourd, A. Finkel, and P. Schnoebelen. Reset nets between decidability and undecidability. In K. Larsen, S. Skyum, and G. Winskel, editors, Proceedings of the 25th International Colloquium on Automata, Languages and Programming (ICALP'98), volume 1443 of Lecture Notes in Computer Science, pages 103–115, Aalborg, Denmark, July 1998. Springer.
- [FRSB02] A. Finkel, J.-F. Raskin, M. Samuelides, and L. van Begin. Monotonic extensions of petri nets: Forward and backward search revisited. Electronic Notes in Theoretical Computer Science, 68(6):1–22, 2002.
- [FS01] A. Finkel and Ph. Schnoebelen. Well-structured transition systems everywhere! Theoretical Computer Science, 256(1–2):63–92, April 2001.
- [JK09] K. Jensen and L.M. Kristensen. Coloured Petri Nets: Modelling and Validation of Concurrent Systems, Springer 2009.
- [Peterson81] J.L.A. Peterson. Petri net theory and the modeling of systems. Prentice Hall, 1981.
- [HAAR09] A.H.M. ter Hofstede, W.M.P. van der Aalst, M. Adams, and N. Russell (editors). Modern Business Process Automation: YAWL and Its Support Environment. Springer, 2010.
- [WfMC] Workflow Management Coalition - Terminology & Glossary, Document number WFMC-TC-1011, Document Status 3.0, February 1999. Downloaded from http://www.aiim.org/wfmc/mainframe.htm. (this document contains the quoted definitions)
- [KHA03] B. Kiepuszewski, A.H.M. ter Hofstede and W.M.P. van der Aalst. Fundamentals of Control Flow in Workflows. Acta Informatica 39(3):143-209, 2003.
- [KHB00] B. Kiepuszewski, A.H.M. ter Hofstede, C. Bussler. On Structured Workflow Modelling. Proceedings CAiSE'2000, Lecture Notes in Computer Science 1789, Stockholm, Sweden, June 2000.
- [Kie03] B. Kiepuszewski. Expressiveness and Suitability of Languages for Control Flow Modelling in Workflows. PhD thesis, Queensland University of Technology, Brisbane, Australia, 2003.
- www.workflowcourse.com (among others for the animations)

Petri Net for a Product Inspection Process (with Rework)

Example: Process Description

- A company manufactures a product and performs quality inspection.
- Start production.
- Manufacture the product.
- Inspect the product.
- If the product **passes inspection**, it is shipped.
- If the product **fails inspection**, it goes to **rework**.
- After rework, the product is **inspected again**.
- This creates a **loop (rework cycle)**.

Petri Net Components

Place	Meaning
P1	Order ready to produce
P2	Product manufactured
P3	Product under inspection
P4	Rework required
P5	Product completed

Transition	Activity
T1	Manufacture product
T2	Start inspection
T3	Inspection passed
T4	Inspection failed
T5	Rework product

Petri Net Structure

Flow of the Petri Net:

- P1 --T1--> P2 --T2--> P3
  P3 --T3--> P5 (pass inspection)
  P3 --T4--> P4 --T5--> P2 (rework loop)

Explanation:

- A **token in P1** means a product is ready to be produced.
- When **T1 fires**, the token moves to **P2**.
- **T2** moves the token to **inspection**.
- At inspection:
  - **T3 fires** → product finished.
  - **T4 fires** → product fails → goes to **rework**.
- After **rework (T5)**, the product returns to **P2** and the cycle repeats.

Initial tokens:

```
P1 = 1
P2 = 0
P3 = 0
P4 = 0
P5 = 0
```

Execution example:

1.T1 fires → token moves to P2
2.T2 fires → token moves to P3
3.If inspection fails → **T4 fires** → token goes to P4
4.T5 fires → token goes back to P2
5.Process repeats until **T3 fires**.

Tools for petrinets

Tool	Type	What it Does	Notes
PetriStation	Web-based	Create and simulate Petri nets collaboratively in the browser	Good for simple modeling and sharing diagrams (PetriStation)
ChatDiagram Petri Net Maker	Web-based	Generates Petri net diagrams from text descriptions	Useful for quick diagrams and exports (Chat Diagram)
PIPE (Platform Independent Petri Net Editor)	Desktop	Create, analyze, and simulate Petri nets with reachability graphs	Popular academic tool, open source (Pipe2)
CPN Tools / CPN IDE	Desktop	Advanced environment for modeling and simulating **Colored Petri Nets**	Used widely in research and process analysis (CPN Tools)
Petri Net Editor (OnWorks)	Online runtime	Java-based editor for building and simulating stochastic Petri nets	Can run in a browser-based virtual environment (OnWorks.net)
WebSPN	Web-based	Tool for analyzing stochastic Petri nets and system performance	Supports state-space and token simulation (University of Hamburg Informatics)

Example Petri Net Process

Place	Meaning
P1	Order ready
P2	Order checked
P3	Product inspected
P4	Order completed

Transition	Activity	Time
T1	Receive order	4 min
T2	Check order	7 min
T3	Inspect product	6 min

$$P1 \rightarrow T1 \rightarrow P2 \rightarrow T2 \rightarrow P3 \rightarrow T3 \rightarrow P4$$

Step 1: Add Processing Times

- In **timed Petri nets**, transitions have firing times.

Transition	Time	Capacity
T1	4 min	0.25 cases/min
T2	7 min	0.143 cases/min
T3	6 min	0.167 cases/min

Capacity=1/Time

Step 2: Observe Token Flow

- Suppose tokens arrive every **4 minutes**.

Execution behavior:

- **T1 fires every 4 min** → tokens move to **P2**
- **T2 needs 7 min** → slower than T1
- Tokens start **accumulating in P2**

Time	P1	P2	P3
0	1	0	0
4	0	1	0
8	0	2	0
12	0	3	1

Visual Interpretation

```
P1 → T1 → P2 → T2 → P3 → T3 → P4
          ↑
     tokens accumulate
```

The queue forms before **T2**.

What Tools Do

A bottleneck transition is the one where: tokens accumulate in its input place, firing rate is lower than incoming rate, it limits the throughput of the net

- Petri net tools (like **PIPE or CPN Tools**) determine bottlenecks by measuring:
  - transition utilization
  - token waiting time
  - queue length in places
  - firing frequency
- The transition with **highest utilization or largest queue before it** is the bottleneck.